

Prctica 12

Cesar zaid reynoso mtz

Jesus collazo

Programacion orientada a objetos

Introduccion

Que es el el polimorfismo

es la capacidad de objetos de distintos tipos, pero con una interfaz común, de comportarse de maneras diferentes ante una misma orden o método. Es un pilar fundamental de la programación orientada a objetos (POO) que permite mejorar la reutilización y flexibilidad del código al tratar objetos derivados como objetos de una clase base

En el contexto de esta práctica, se simula un sistema de combate en un videojuego donde existen distintos tipos de personajes. Aunque todos pueden realizar la acción de atacar, cada uno lo hace de manera diferente. Gracias al polimorfismo, el programa puede manejar a todos los personajes de forma general sin necesidad de saber exactamente qué tipo de personaje es, permitiendo así ejecutar la misma instrucción y obtener comportamientos distintos.

En conclusión, el polimorfismo sirve para simplificar el código, hacerlo más escalable y permitir la extensión del sistema sin necesidad de modificar su estructura principal, lo cual es especialmente útil en el desarrollo de videojuegos y sistemas complejos.

Personaje.h

El archivo .h se utiliza para definir la estructura del programa. En este archivo se declaran las clases, sus atributos y los métodos que van a tener, pero sin incluir la implementación completa de estos.

Personaje.cpp

En esta práctica, Personaje.cpp contiene la lógica de los métodos como mostrarInfo () y atacar (). Además, se implementa el comportamiento específico de cada clase derivada (Guerrero, Mago y Arquero), donde cada uno redefine el método atacar () de manera distinta. Esto permite que, al ejecutar el programa, cada personaje responda de forma diferente, aplicando así el polimorfismo.

Main.cpp

En esta práctica, en el main se crean diferentes personajes como Guerrero, Mago y Arquero, y se almacenan en un vector de punteros a la clase base Personaje. Después, se recorre la colección para mostrar la información de cada personaje y ejecutar el método atacar ().

Gracias al polimorfismo, aunque todos los objetos se manejan como tipo Personaje, cada uno ejecuta su propio comportamiento al atacar, lo que demuestra que el sistema puede trabajar con distintos tipos de personajes de forma genérica

Explicación

En esta práctica, el polimorfismo se implementó mediante una clase base llamada Personaje y varias clases derivadas como Guerrero, Mago y Arquero. Todas comparten el mismo método atacar (), pero cada una lo redefine para tener un comportamiento diferente. Esto permite que el programa ejecute la misma instrucción y obtenga resultados distintos dependiendo del objeto.

Una función virtual es un método declarado en la clase base que puede ser sobrescrito en las clases derivadas. En este caso, atacar () se declara como virtual para que cada personaje pueda modificar su comportamiento al momento de ejecutarlo.

El uso de punteros a la clase base permite almacenar distintos tipos de objetos en una misma estructura, como un vector de Personaje*. La ventaja de esto es que el programa puede manejar todos los personajes de forma genérica sin necesidad de saber su tipo específico, lo que hace el código más flexible y fácil de extender

CONCLUSION

En esta práctica logré comprender cómo se aplica el polimorfismo dentro de la Programación Orientada a Objetos y la manera en que C++ permite reutilizar código mediante clases base y derivadas. Además, aprendí la función de los métodos virtuales y cómo diferentes objetos pueden ejecutar acciones distintas utilizando una misma referencia o puntero.

Al inicio resultó un poco complicado entender la relación entre herencia, sobrescritura de métodos y polimorfismo, ya que era necesario identificar cómo interactúan las clases entre sí. Aun así, conforme avancé en el desarrollo del programa, pude entender mejor la lógica y la estructura necesaria para implementarlo correctamente.

Considero que este tema es muy importante porque ayuda a crear programas más organizados, reutilizables y fáciles de mantener. El polimorfismo es ampliamente utilizado en sistemas grandes, aplicaciones y videojuegos, donde se requiere manejar diferentes tipos de objetos de una forma más eficiente y flexible.

1. ¿Qué problema se intenta resolver con el polimorfismo?

Se busca que distintos tipos de personajes puedan ejecutar la misma acción (atacar) sin importar su tipo, evitando usar condicionales como if o switch.

2. ¿Qué clase representaría el comportamiento común?

Una clase base llamada Personaje.

3. ¿Qué método sería ideal para polimorfismo?

El método atacar().

4. ¿Por qué debe ser virtual?

Para permitir que las clases hijas redefinan su propio comportamiento.

5. ¿Qué pasaría si NO fuera virtual?

Todos los personajes usarían el mismo ataque, perdiendo el comportamiento específico.

6. ¿Cómo lograr comportamiento distinto?

Cada clase hija (Guerrero, Mago, Arquero) sobrescribe el método atacar().

7. ¿Cómo ejecutar ataques sin saber el tipo?

Usando punteros o referencias a la clase base (Personaje).

8. ¿Qué estructura usarías?

Una colección como un arreglo o lista de punteros a Personaje.

9. ¿Ventaja de usar punteros a clase base?

Permite tratar todos los personajes de forma uniforme.

10. ¿Tiempo de ejecución?

El método que se ejecuta se decide cuando el programa está corriendo, no al compilar.

11. ¿Cómo evita condicionales?

Cada objeto sabe qué hacer, no necesitas preguntar su tipo.

12. ¿Si agregas “Hechicero”?

Solo creas una nueva clase que herede de Personaje

.

13. ¿Se modifica el sistema?

No, porque el sistema ya está preparado para nuevos tipos.

14. Ventajas en videojuegos

Escalabilidad

Código limpio

Fácil mantenimiento

15. Relación herencia-polimorfismo

La herencia permite crear clases hijas y el polimorfismo permite que actúen diferente.

16. ¿Interfaz común?

Es la clase base (Personaje) que define métodos comunes como atacar().

17. Problemas sin polimorfismo

Código repetido

Uso excesivo de if

Difícil mantenimiento

18. ¿Agregar más acciones?

Crear más métodos virtuales como defender(), curar(), etc.

19. ¿Cuándo complica?

Cuando hay demasiadas clases o jerarquías mal diseñadas.

20. Explicación sencilla

El polimorfismo permite que diferentes personajes hagan lo mismo (atacar), pero cada uno a su manera

